Ускорение инференса UNet

для задачи Super-Resolution ×4

Efficient ML — Team 52

Артём Чубов · Женя Веселков · Андрей Петухов

Апрель 2026

Задача: Super-Resolution ×4

LR вход (180×140 px)

SR выход (720×560 px)

Что делает модель

По Low-Resolution входу восстанавливает детали — масштаб ×4

LR 256×256 → SR 256×256

(вычисления на LR-разрешении)

Датасет: DIV2K

2K HR-фотографии природы

Train	800 изображений
Val	50 изображений
Test	50 изображений

LR on-the-fly: bicubic ↓×4 → ↑×4

Модель: SRUNetHeavy

Архитектура

- U-Net, 5 уровней энкодер/декодер
- **MBResBlock** (inverted residual): Conv1×1 → DWConv3×3 → Conv1×1
- **SEBlock** в боттлнеке
- Глобальный residual
- **GroupNorm** — каналы можно удалять без пересчёта статистик

Ключевые параметры

Параметры	**29.3M**
FLOPs	138 GFLOPs
Вход/выход	256×256 → 256×256
Метрика	SSIM (FP32 ref = **0.7689**)
Бенчмарк	CUDA Events, BS=1

Метрика ускорения

Speedup = *baseline latency / result latency*

Контроль качества: SSIM vs FP32 baseline

Главные результаты — все методы

Метод	Precision	Speedup	SSIM	GPU
Eager baseline	FP32	1.00×	0.769	RTX 3060
Eager baseline	FP16	1.52×	0.769	RTX 3060
torch.compile	FP32	1.03×	0.769	RTX 3060
ORT TensorRT EP	FP32	1.61×	0.769	RTX 3060
ORT TRT EP	**FP16**	**3.62×**	**0.769**	RTX 3060
LSQ PTQ + FP16	INT8/FP16	≈1.0× *	0.768	A100
Struct. pruning 25%	FP32	1.23×	0.760	RTX 8000
Struct. pruning 50%	FP32	1.57×	0.758	RTX 8000
Struct. pruning 75%	**FP32**	**2.27×**	**0.760**	RTX 8000
2:4 sparsity (FP16)	FP16	0.84×	—	A100

* LSQ PTQ: главный выигрыш — сжатие модели в **4×** (112 MB → 28 MB)

Артём: Квантизация LSQ — метод

LSQ: Learned Step Size Quantization

Fake quantization — форвард симулирует INT8,
веса остаются в float:

$$\hat{w} = \text{round}(w \,/\, s) \cdot s$$

- s — learnable per-channel step size
- Градиент через `round`: STE
- **PTQ:** s из статистик весов, finetune не нужен

`LsqConv2d/LsqLinear` — drop-in замена
`apply_lsq_ptq()` — калибровка на val
`finalize_lsq()` — экспорт INT8 + FP16 scales

Сжатие модели

Формат	Размер	SSIM
FP32	111.7 MB	0.7689
FP16	55.8 MB	0.7689
LSQ INT8	**28.1 MB**	**0.7680**

4× меньше FP32
SSIM $\Delta = -0.0009$ (без finetune)

INT8 state dict: веса в `int8` + FP16 scales

Артём: Квантизация LSQ — результаты

NVIDIA A100-SXM4-80GB · 256×256 · 100 runs

Config	BS=1	BS=4	BS=16	BS=64
FP32	30.2 ms	51.0 ms	128.9 ms	485.8 ms
FP16	26.1 ms	41.0 ms	89.3 ms	306.9 ms
LSQ+FP32	41.3 ms	61.5 ms	139.5 ms	496.0 ms
LSQ+FP16	37.7 ms	51.9 ms	99.9 ms	**316.8 ms**

FP32 111.7 MB
FP16 55.8 MB
INT8 28.1 MB

Выводы

- LSQ+FP16 при BS=64 → **0.97×** vs FP16
- Fake quant не ускоряет: вычисления в float
- Выигрыш: **хранение** (4×)
- Для speedup нужны INT8-ядра (TRT INT8, torch.ao)

Женя: Baseline и компиляция

SRUNetHeavy 29M · RTX 3060 · BS=1 · 50 runs

Метод	Prec.	Latency	Speedup
Eager PyTorch	FP32	48.1 ms	1.00×
Eager PyTorch	FP16	31.7 ms	1.52×
torch.compile	FP32	46.6 ms	1.03×
ORT CUDA EP	FP32	46.1 ms	1.04×
ORT TRT EP	FP32	29.8 ms	1.61×
ORT TRT EP	**FP16**	**13.3 ms**	**3.62×**

Throughput TRT FP16: **75.2 img/s** vs 20.8 img/s (Eager FP32)

torch.compile — почему мало?

Triton не находит fusion для `GroupNorm + GELU + DWConv3×3`

TensorRT кодирует их custom plugins → kernel launch overhead ↓↓

TRT FP16 — откуда 3.62×?

- 1×1 Conv = GEMM → Tensor Cores
- Fusion GroupNorm+GELU+Conv
- Вдвое меньше DRAM-трафика

Женя: TensorRT — сравнение моделей

Модель	Метод	Latency	Speedup
SRUNetHeavy	Eager FP32	48.1 ms	1.00×
	TRT FP32	29.8 ms	1.61×
	TRT FP16	**13.3 ms**	**3.62×**
UNetSR	Eager FP32	119.7 ms	1.00×
	TRT FP32	90.6 ms	1.32×
	TRT FP16	**30.8 ms**	**3.87×**

Почему SRUNetHeavy лучше реагирует на TRT FP32?

×1.61 vs ×1.32 у UNetSR

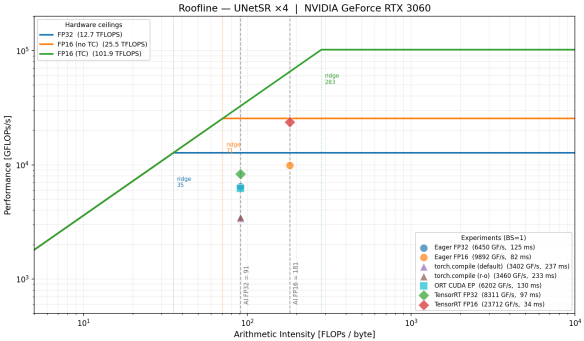
MBResBlock: больше 1×1 conv относительно DWConv → fusion GroupNorm+GELU+Conv1×1 важнее

Ограничение DWConv

DWConv3×3 **не использует Tensor Cores** (TC нужны матрицы ≥16×16)

FP16 ускоряется за счёт bandwidth, а не вычислений

Женя: Roofline-анализ



Arithmetic Intensity

	AI	режим
FP32 ridge	35	—
FP16 TC ridge	283	—
AI FP32	210	compute-bound
AI FP16	420	compute-bound

Почему лишь 22% от пика?

DWConv3×3 — каналы независимо, warp occupancy низкий

Eager FP32: 22% пика
TRT FP32: 36% пика
TRT FP16: 10% TC-пика

Андрей: Структурное прунирование — идея

Expand-Channel Pruning

В каждом MBResBlock физически удаляем каналы expand 1×1 conv с наименьшей L1-нормой

U-Net топология не нарушается: только `mid_ch` уменьшается

Почему даёт реальный speedup

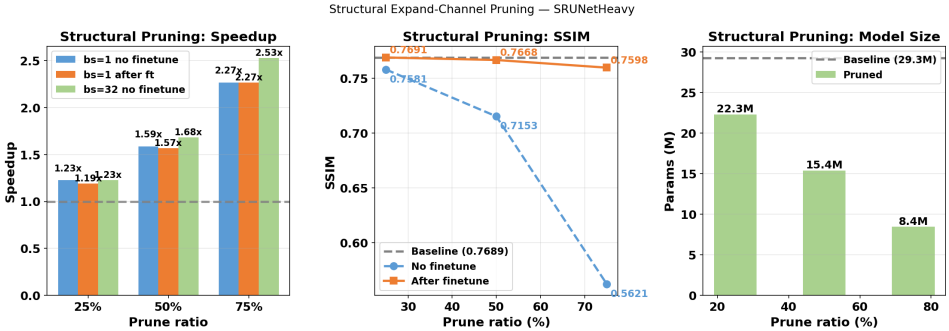
- Тензоры меньше физически — не нужны sparse kernels
- GroupNorm — каналы удаляются без пересчёта
- Finetune (10 эпох) восстанавливает SSIM

Результаты: 75% + finetune

	До	После
Параметры	29.3M	**8.4M**
Latency BS=1	34.2 ms	**15.1 ms**
Speedup BS=1	—	**2.27×**
SSIM	0.7689	0.7598

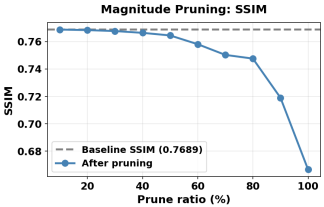
Модель в **3.5×** меньше · SSIM **−0.009**

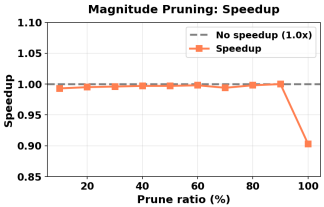
Андрей: Структурное прунирование — результаты



Прунирование	Speedup BS=1	Speedup BS=32	SSIM (ft)	Params
25% + finetune	1.23×	1.23×	0.760	22.3M
50% + finetune	1.57×	1.68×	0.758	15.4M
75% + finetune	**2.27×**	**2.53×**	**0.760**	**8.4M**

Андрей: Magnitude Pruning





Метод

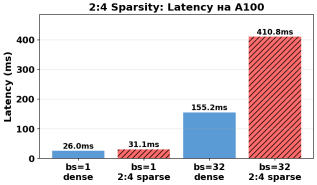
Обнуляем N% наименьших весов 1×1 Conv2d через `global_unstructured`

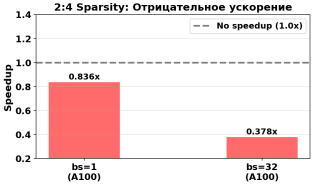
Speedup ≈ 1.0×

Тензоры остаются **dense** — GPU исполняет те же CUDA-ядра с теми же матрицами

Нули перемножаются, а не пропускаются

Андрей: 2:4 Semi-structured Sparsity





Метод

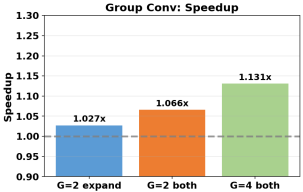
В каждой группе из 4 весов ровно 2 обнуляются (по наименьшей норме).

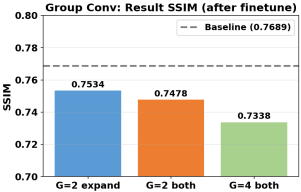
`to_sparse_semi_structured` + cuSPARSELt

Почему нет speedup

- cuSPARSELt — только **Ampere (sm_80+)**, RTX 8000 = Turing
- На A100: матрицы 1×1 Conv слишком малы — overhead перевешивает
- BS=1: **0.84×**, BS=32: **0.38×**

Андрей: Grouped Convolutions





Метод

expand/project 1×1 Conv → grouped conv (G=2 или G=4).

FLOPs ÷ G, но CUDA matmul overhead ограничивает реальный speedup.

Сравнение

G=4 both: **1.13×**, SSIM -0.035

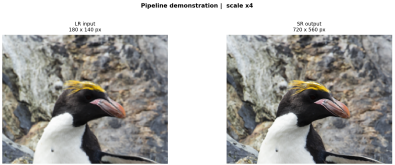
Struct. 75%: **2.27×**, SSIM -0.009

→ Структурное прунирование превосходит по обеим метрикам

Выводы

Лучшие результаты

Цель	Метод	Итог
Макс. ускорение	TRT FP16	**3.62×**
Без потерь SSIM	TRT FP16	SSIM = 0.769
Сжатие модели	LSQ PTQ	**4×**
Speed+компактность	Struct. 75%	**2.27×**



Ключевые наблюдения

- **torch.compile** — минимальный выигрыш:
  Triton не фьюзит GroupNorm+GELU+DWConv
- **TensorRT** — лучший бэкенд: fusion + Tensor Cores
- **LSQ PTQ** — 4×меньше, но без INT8-ядер не ускоряет
- **Struct. pruning** — лучший трейдофф, любой GPU
- **2:4 sparsity** — нужен Ampere+, малые матрицы убыточны